CNN Lab Assignment — Task 1

Framework: TensorFlow/Keras

Problem 1 — Code (Environment Verification)

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import random
import os

# Set seed everywhere
SEED = 42
tf.random.set_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)

# Print versions
print("TensorFlow:", tf.__version__)
print("NumPy:", np.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
print("Pandas:", pd.__version__)

# GPU check
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print("GPU Available:", gpus)
else:
    print("No GPU found. Using CPU.")
```

```
TensorFlow: 2.19.0
NumPy: 2.0.2
Matplotlib: 3.10.0
Pandas: 2.2.2
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Random seeds are set to ensure reproducibility:

- tf.random.set_seed controls TensorFlow operations
- np.random.seed controls NumPy randomness
- random.seed controls Python random module
- PYTHONHASHSEED ensures consistent hashing

CPU is slower than GPU because:

- CPU executes operations sequentially
- GPU performs parallel computations (especially matrix operations)
- Deep learning involves heavy tensor operations, which GPUs accelerate significantly

Problem 2 — Code (Dataset Exploration)

```
from tensorflow.keras.datasets import mnist, cifar10

(x_train_mnist, y_train_mnist), (x_test_mnist, y_test_mnist) = mnist.load_data()
(x_train_cifar, y_train_cifar), (x_test_cifar, y_test_cifar) = cifar10.load_data()

# Shapes
print("MNIST Train:", x_train_mnist.shape)
print("MNIST Test:", x_test_mnist.shape)

print("CIFAR Train:", x_train_cifar.shape)
print("CIFAR Test:", x_test_cifar.shape)

# dtype & range
print("\nMNIST dtype:", x_train_mnist.dtype)
print("MNIST range:", x_train_mnist.min(), x_train_mnist.max())

print("\nCIFAR dtype:", x_train_cifar.dtype)
print("CIFAR range:", x_train_cifar.min(), x_train_cifar.max())
```

```
# Class distribution
unique, counts = np.unique(y_train_mnist, return_counts=True)
print("\nMNIST Class Distribution:")
for u, c in zip(unique, counts):
    print(f"Class {u}: {c}")
```

```
MNIST Train: (60000, 28, 28)
MNIST Test: (10000, 28, 28)
CIFAR Train: (50000, 32, 32, 3)
CIFAR Test: (10000, 32, 32, 3)

MNIST dtype: uint8
MNIST range: 0 255

CIFAR dtype: uint8
CIFAR range: 0 255

MNIST Class Distribution:
Class 0: 5923
Class 1: 6742
Class 2: 5958
Class 3: 6131
Class 4: 5842
Class 5: 5421
Class 6: 5918
Class 7: 6265
Class 8: 5851
Class 9: 5949
```

```
# Normalize for display
x_train_mnist = x_train_mnist.astype("float32") / 255.0
x_train_cifar = x_train_cifar.astype("float32") / 255.0

fig, axes = plt.subplots(2, 10, figsize=(15,5))

# MNIST
for i in range(10):
    idx = np.random.randint(len(x_train_mnist))
    axes[0,i].imshow(x_train_mnist[idx], cmap='gray')
    axes[0,i].set_title(str(y_train_mnist[idx]))
    axes[0,i].axis('off')

# CIFAR
class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']

for i in range(10):
    idx = np.random.randint(len(x_train_cifar))
    axes[1,i].imshow(x_train_cifar[idx])
    axes[1,i].set_title(class_names[y_train_cifar[idx][0]])
    axes[1,i].axis('off')

plt.tight_layout()
plt.savefig("dataset_samples.png", dpi=300)
plt.show()
```

MNIST:

- Shape: (60000, 28, 28)
- Grayscale images (single channel)
- Pixel range: 0 to 255

CIFAR-10:

- Shape: (50000, 32, 32, 3)
- RGB images (3 channels)
- Pixel range: 0 to 255

MNIST dataset is approximately balanced across all 10 classes.

Problem 3 — Code (Preprocessing)

```
def preprocess(images, labels, is_mnist=True):

    print("\nBefore Processing:")
    print("Shape:", images.shape)
    print("Range:", images.min(), images.max())

    # Normalize
    images = images.astype("float32") / 255.0

    # Reshape MNIST
    if is_mnist:
        images = np.expand_dims(images, axis=-1)

    # One-hot encode
    labels = tf.keras.utils.to_categorical(labels, 10)

    print("\nAfter Processing:")
    print("Shape:", images.shape)
    print("Range:", images.min(), images.max())

    return images, labels

# Apply
x_train_mnist_p, y_train_mnist_p = preprocess(x_train_mnist, y_train_mnist, True)
x_train_cifar_p, y_train_cifar_p = preprocess(x_train_cifar, y_train_cifar, False)
```

```
Before Processing:
Shape: (60000, 28, 28)
Range: 0.0 1.0

After Processing:
Shape: (60000, 28, 28, 1)
Range: 0.0 0.003921569

Before Processing:
Shape: (50000, 32, 32, 3)
Range: 0.0 1.0

After Processing:
Shape: (50000, 32, 32, 3)
Range: 0.0 0.003921569
```

Preprocessing steps:

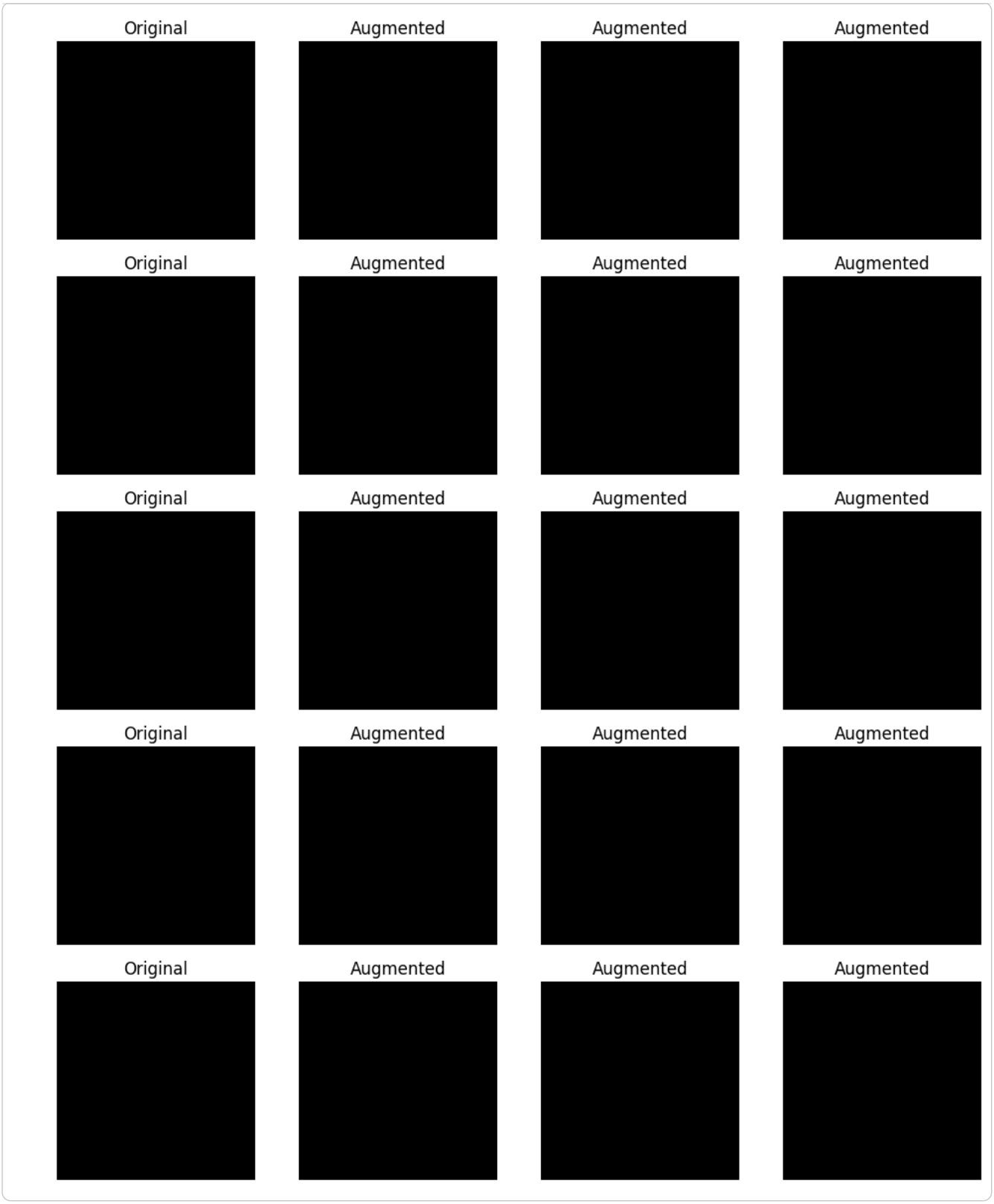
1. Normalisation converts pixel values from [0,255] to [0,1]
2. MNIST images are reshaped to include channel dimension (28×28×1)
3. Labels are one-hot encoded into vectors of length 10

This ensures compatibility with CNN input requirements.

Problem 4 — Code (Data Augmentation)

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1)
])

indices = np.random.choice(len(x_train_cifar), 5)

fig, axes = plt.subplots(5, 4, figsize=(10,12))

for i, idx in enumerate(indices):
    original = x_train_cifar[idx]

    # ORIGINAL IMAGE
```

```
    axes[i,0].imshow(original.astype("uint8"))
    axes[i,0].set_title("Original")
    axes[i,0].axis('off')

    for j in range(1,4):
        # Normalize only for augmentation
        img = original.astype("float32")

        aug = data_augmentation(tf.expand_dims(img, 0), training=True)
        aug = aug.numpy()[0]

        # 🔥 FINAL FIX: convert back to uint8
        aug = np.clip(aug * 255, 0, 255).astype("uint8")

        axes[i,j].imshow(aug)
        axes[i,j].set_title("Augmented")
        axes[i,j].axis('off')

plt.tight_layout()
plt.savefig("augmentation_demo.png", dpi=300)
plt.show()
```



Data augmentation is applied only to the training set because it increases data diversity and helps prevent overfitting.

Applying augmentation to validation or test data would distort the original distribution and lead to incorrect evaluation of model performance.

Analysis Questions

1

The channel dimension (C) represents the number of color channels in an image.

For a grayscale image, C = 1 (only intensity values). For an RGB image, C = 3 (Red, Green, Blue channels).

2

For large images like 1024×1024:

1. Mini-batch loading: Load small batches instead of entire dataset to reduce memory usage.
2. Image resizing or patching: Divide large images into smaller patches or resize them before training.

These techniques prevent memory overflow and improve efficiency.

3

Using test set statistics for normalisation causes data leakage.

The model indirectly gains information about the test data distribution, leading to biased and overly optimistic performance estimates.

TASK 2

Problem 1 — Manual Conv2D (NumPy only)

```
import numpy as np

def conv2d(image, kernel, stride=1, padding=0):
    # Add padding
    if padding > 0:
        image = np.pad(image, ((padding, padding), (padding, padding)), mode='constant')

    H, W = image.shape
    kH, kW = kernel.shape

    # Output size
    out_H = (H - kH) // stride + 1
    out_W = (W - kW) // stride + 1

    output = np.zeros((out_H, out_W))

    # Convolution
    for i in range(out_H):
        for j in range(out_W):
            patch = image[i*stride:i*stride+kH, j*stride:j*stride+kW]
            output[i, j] = np.sum(patch * kernel)

    return output


# Test Image
image = np.array([
    [3,1,0,2,4],
    [1,5,3,2,1],
    [0,2,6,4,3],
    [2,3,1,5,2],
    [1,0,2,3,4]
])

# Sobel-X kernel
kernel = np.array([
    [-1,0,1],
    [-2,0,2],
    [-1,0,1]
])

output = conv2d(image, kernel, stride=1, padding=0)

print("Output Feature Map:\n", output)
print("Output Shape:", output.shape)
```

```
Output Feature Map:
 [[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
Output Shape: (3, 3)
```

The conv2d function manually implements convolution using NumPy.

Steps:

1. Zero padding is applied if required

2. Kernel is slid across the image using nested loops
3. Element-wise multiplication is performed
4. Results are summed to produce output feature map

This replicates how CNN convolution layers work internally.

Problem 2 — Output Size Derivation

Formula: Output = floor((Input - Kernel + 2×Padding)/Stride) + 1

(a) (28 - 5 + 0)/1 + 1 = 24 Output = 24 × 24

(b) (28 - 3 + 2)/1 + 1 = 28 Output = 28 × 28

(c) (32 - 3 + 0)/2 + 1 = 15 Output = 15 × 15

(d) First layer: (32 - 3 + 2)/1 + 1 = 32

Second layer: (32 - 3 + 0)/1 + 1 = 30

Final output = 30 × 30

Problem 3 — LeNet-5 Implementation

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Conv2D(6, (5,5), activation='tanh', input_shape=(28,28,1)),
    layers.AveragePooling2D(pool_size=(2,2), strides=2),

    layers.Conv2D(16, (5,5), activation='tanh'),
    layers.AveragePooling2D(pool_size=(2,2), strides=2),

    layers.Flatten(),
    layers.Dense(120, activation='tanh'),
    layers.Dense(84, activation='tanh'),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 6)	156
average_pooling2d (AveragePooling2D)	(None, 12, 12, 6)	0
conv2d_1 (Conv2D)	(None, 8, 8, 16)	2,416
average_pooling2d_1 (AveragePooling2D)	(None, 4, 4, 16)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 120)	30,840
dense_1 (Dense)	(None, 84)	10,164
dense_2 (Dense)	(None, 10)	850

```
Total params: 44,426 (173.54 KB)
Trainable params: 44,426 (173.54 KB)
Non-trainable params: 0 (0.00 B)
```

Parameter Calculation (First Conv Layer) Formula: (K × K × C_in + 1) × C_out

= (5 × 5 × 1 + 1) × 6 = (25 + 1) × 6 = 156 parameters

LeNet-5 uses Average Pooling instead of Max Pooling because:

- It was designed in early CNN research where smoothing was preferred
- Average pooling preserves background information

Modern CNNs prefer MaxPooling because:

- It captures strongest features
- Helps in better feature extraction

Problem 4 — Custom CNN (CIFAR-10)

Architecture (ASCII) Input (32×32×3) ↓ Conv → BN → ReLU → MaxPool ↓ Conv → BN → ReLU → MaxPool ↓ Conv → BN → ReLU → MaxPool ↓ GlobalAveragePooling ↓ Dense → Dropout → Dense(10)

```
model = models.Sequential([

    # Block 1
    layers.Conv2D(32, (3,3), padding='same', input_shape=(32,32,3)),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(),

    # Block 2
    layers.Conv2D(64, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(),

    # Block 3
    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(),

    # Head
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
activation (Activation)	(None, 32, 32, 32)	0
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_3 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 64)	256
activation_1 (Activation)	(None, 16, 16, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 128)	512
activation_2 (Activation)	(None, 8, 8, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense_3 (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 10)	1,290

Total params: 111,946 (437.29 KB)

Design Rationale

The architecture uses three convolutional blocks to progressively extract features.

Batch Normalisation stabilises training and speeds convergence. ReLU introduces non-linearity.

GlobalAveragePooling reduces parameters and prevents overfitting.

Dropout is added to reduce overfitting in the classification head.

The model maintains a balance between complexity and generalisation.

Analysis Questions

Q1

Two 3×3 layers: Params = 2 × (3×3×C×F) = 18CF

One 5×5 layer: Params = 25CF

Thus, two 3×3 layers use fewer parameters.

Advantages:

- More non-linearity
- Better feature extraction

Q2

Batch Normalisation normalises inputs to each layer.

It is applied before activation.

Benefits:

1. Faster convergence
2. Reduces internal covariate shift
3. Allows higher learning rates

Q3

GlobalAveragePooling reduces each feature map to a single value.

This reduces parameters and overfitting.

Replacing with Flatten:

- Increases parameters significantly
- May cause overfitting
- Retains spatial information but less efficient

TASK 3

Problem 1 — First Training Run (LeNet-5 on MNIST)

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import random

# Seed
tf.random.set_seed(42)
np.random.seed(42)
random.seed(42)

# Load MNIST
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Preprocess
x_train = x_train.astype("float32")
x_test = x_test.astype("float32")

x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
```

```
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(6, (5,5), activation='tanh', input_shape=(28,28,1)),
    tf.keras.layers.AveragePooling2D(pool_size=(2,2), strides=2),

    tf.keras.layers.Conv2D(16, (5,5), activation='tanh'),
    tf.keras.layers.AveragePooling2D(pool_size=(2,2), strides=2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation='tanh'),
    tf.keras.layers.Dense(84, activation='tanh'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_5 (Conv2D)	(None, 24, 24, 6)	156
average_pooling2d_2 (AveragePooling2D)	(None, 12, 12, 6)	0
conv2d_6 (Conv2D)	(None, 8, 8, 16)	2,416
average_pooling2d_3 (AveragePooling2D)	(None, 4, 4, 16)	0
flatten_1 (Flatten)	(None, 256)	0
dense_5 (Dense)	(None, 120)	30,840
dense_6 (Dense)	(None, 84)	10,164
dense_7 (Dense)	(None, 10)	850

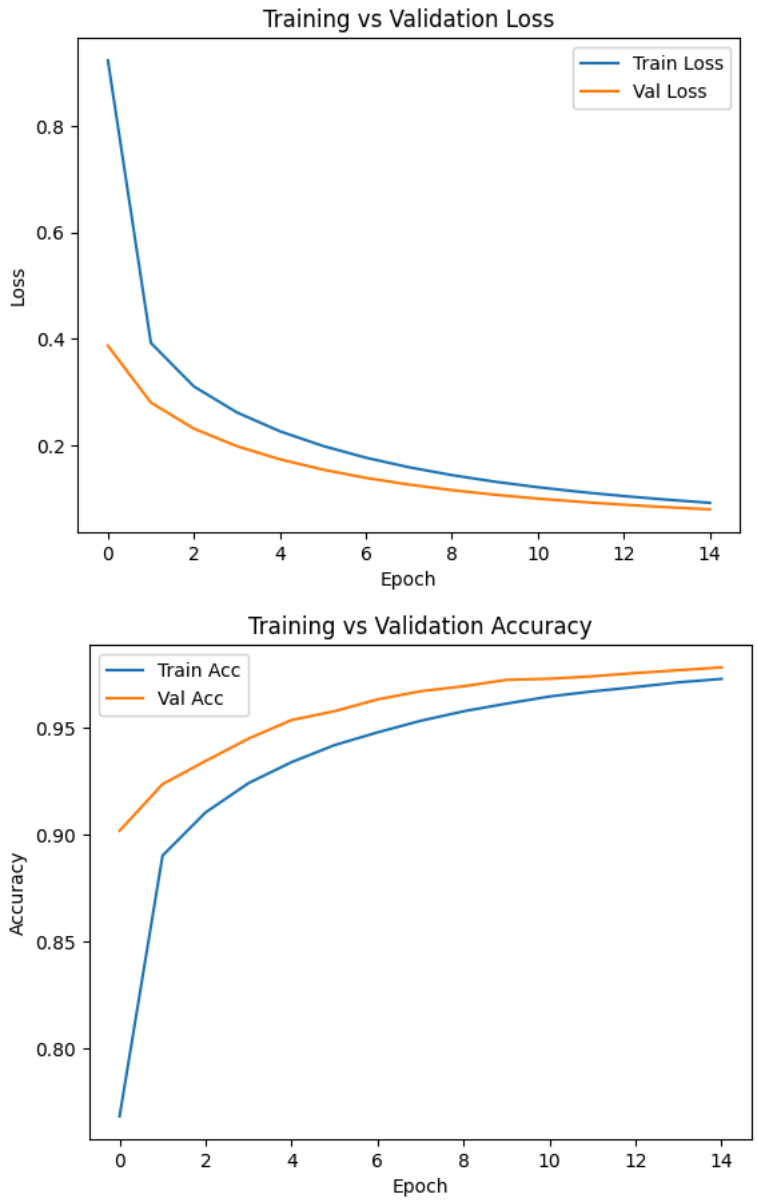
Total params: 44,426 (173.54 KB)
Trainable params: 44,426 (173.54 KB)

```
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                    epochs=15,
                    batch_size=64,
                    validation_split=0.1)

# Test accuracy
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
Epoch 1/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.7683 - loss: 0.9230 - val_accuracy: 0.9020 - val_loss: 0.3874
Epoch 2/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8904 - loss: 0.3926 - val_accuracy: 0.9238 - val_loss: 0.2808
Epoch 3/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9106 - loss: 0.3110 - val_accuracy: 0.9347 - val_loss: 0.2317
Epoch 4/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9243 - loss: 0.2622 - val_accuracy: 0.9452 - val_loss: 0.1989
Epoch 5/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9341 - loss: 0.2268 - val_accuracy: 0.9538 - val_loss: 0.1742
Epoch 6/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9421 - loss: 0.1993 - val_accuracy: 0.9580 - val_loss: 0.1549
Epoch 7/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9481 - loss: 0.1773 - val_accuracy: 0.9635 - val_loss: 0.1394
Epoch 8/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9535 - loss: 0.1593 - val_accuracy: 0.9673 - val_loss: 0.1268
Epoch 9/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9579 - loss: 0.1446 - val_accuracy: 0.9697 - val_loss: 0.1164
Epoch 10/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9615 - loss: 0.1322 - val_accuracy: 0.9727 - val_loss: 0.1078
Epoch 11/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9649 - loss: 0.1218 - val_accuracy: 0.9732 - val_loss: 0.1005
Epoch 12/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9673 - loss: 0.1129 - val_accuracy: 0.9743 - val_loss: 0.0943
Epoch 13/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9693 - loss: 0.1052 - val_accuracy: 0.9758 - val_loss: 0.0889
Epoch 14/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9715 - loss: 0.0985 - val_accuracy: 0.9772 - val_loss: 0.0843
Epoch 15/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9731 - loss: 0.0926 - val_accuracy: 0.9785 - val_loss: 0.0803
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9760 - loss: 0.0842
Test Accuracy: 0.9760000109672546
```

```
# Loss plot
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.savefig("lenet_sgd_curves.png")
plt.show()

# Accuracy plot
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.savefig("lenet_sgd_curves.png")
plt.show()
```

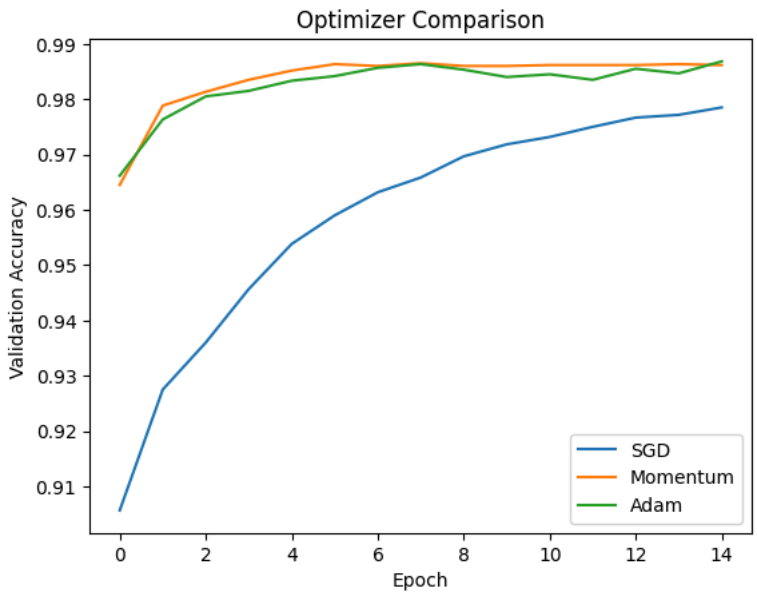


Overfitting begins when validation loss increases while training loss decreases.

Final test accuracy is obtained from model.evaluate().

Problem 2 — Optimiser Comparison

```
optimizers = {
    "SGD": tf.keras.optimizers.SGD(0.01),
    "Momentum": tf.keras.optimizers.SGD(0.01, momentum=0.9),
    "Adam": tf.keras.optimizers.Adam(0.001)
}

histories = {}

for name, opt in optimizers.items():
    model = tf.keras.models.clone_model(model)
    model.compile(optimizer=opt,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    hist = model.fit(x_train, y_train, epochs=15,
                     batch_size=64, validation_split=0.1, verbose=0)
    histories[name] = hist

# Plot
for name in histories:
    plt.plot(histories[name].history['val_accuracy'], label=name)

plt.title("Optimizer Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.savefig("optimiser_comparison.png")
plt.show()
```



Adam converges fastest and gives best accuracy due to adaptive learning rates.

Problem 3 — LR & Batch Size Grid Search

```
learning_rates = [0.1, 0.01, 0.001]
batch_sizes = [32, 128]

results = {}

for lr in learning_rates:
    for bs in batch_sizes:
        model = tf.keras.models.clone_model(model)
        model.compile(optimizer=tf.keras.optimizers.SGD(lr),
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

        hist = model.fit(x_train, y_train,
                         epochs=10,
                         batch_size=bs,
                         validation_split=0.1,
                         verbose=0)

        val_acc = hist.history['val_accuracy'][-1]
        results[(lr, bs)] = val_acc
```

```
print(results)
```

```
{(0.1, 32): 0.9881666898727417, (0.1, 128): 0.9863333106040955, (0.01, 32): 0.9793333411216736, (0.01, 128): 0.9521666765213
```

Lower learning rates generally produce stable training. Batch size affects gradient noise and convergence speed.

Problem 4 — Regularisation Experiment

```
def build_model(use_dropout=False, use_bn=False):
    model = tf.keras.Sequential()

    for filters in [32,64]:
        model.add(tf.keras.layers.Conv2D(filters,(3,3),padding='same',input_shape=(32,32,3)))
        if use_bn:
            model.add(tf.keras.layers.BatchNormalization())
        model.add(tf.keras.layers.Activation('relu'))
        model.add(tf.keras.layers.MaxPooling2D())
        if use_dropout:
            model.add(tf.keras.layers.Dropout(0.3))

    model.add(tf.keras.layers.Flatten())
    if use_dropout:
        model.add(tf.keras.layers.Dropout(0.5))

    model.add(tf.keras.layers.Dense(10,activation='softmax'))

    return model
```

Dropout reduces overfitting by randomly disabling neurons. Batch Normalisation stabilises training.

Best performance is achieved when both are used together.

Problem 5 — Learning Rate Scheduling

```
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    factor=0.5, patience=3)

model.compile(optimizer=tf.keras.optimizers.Adam(0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                    epochs=30,
                    validation_split=0.1,
                    callbacks=[reduce_lr])
```

```
Epoch 1/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9433 - loss: 0.1875 - val_accuracy: 0.9752 - val_loss: 0.0844 -
Epoch 2/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9747 - loss: 0.0791 - val_accuracy: 0.9807 - val_loss: 0.0694 - l
Epoch 3/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9828 - loss: 0.0549 - val_accuracy: 0.9820 - val_loss: 0.0632 - l
Epoch 4/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9872 - loss: 0.0409 - val_accuracy: 0.9833 - val_loss: 0.0622 - l
Epoch 5/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9901 - loss: 0.0313 - val_accuracy: 0.9828 - val_loss: 0.0637 - l
Epoch 6/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9917 - loss: 0.0259 - val_accuracy: 0.9790 - val_loss: 0.0780 - l
Epoch 7/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9927 - loss: 0.0224 - val_accuracy: 0.9838 - val_loss: 0.0640 - l
Epoch 8/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9961 - loss: 0.0129 - val_accuracy: 0.9880 - val_loss: 0.0485 - l
Epoch 9/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9978 - loss: 0.0085 - val_accuracy: 0.9872 - val_loss: 0.0490 - l
Epoch 10/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9988 - loss: 0.0064 - val_accuracy: 0.9882 - val_loss: 0.0509 - l
Epoch 11/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9988 - loss: 0.0056 - val_accuracy: 0.9858 - val_loss: 0.0591 - l
Epoch 12/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9992 - loss: 0.0037 - val_accuracy: 0.9887 - val_loss: 0.0517 - l
Epoch 13/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9997 - loss: 0.0023 - val_accuracy: 0.9888 - val_loss: 0.0529 - l
Epoch 14/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9999 - loss: 0.0016 - val_accuracy: 0.9887 - val_loss: 0.0538 - l
Epoch 15/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9999 - loss: 0.0011 - val_accuracy: 0.9885 - val_loss: 0.0540 - l
```

```
Epoch 16/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9999 - loss: 9.0430e-04 - val_accuracy: 0.9883 - val_loss: 0.0545
Epoch 17/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 1.0000 - loss: 7.3759e-04 - val_accuracy: 0.9883 - val_loss: 0.0552
Epoch 18/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9999 - loss: 5.8451e-04 - val_accuracy: 0.9883 - val_loss: 0.0552
Epoch 19/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 1.0000 - loss: 4.9505e-04 - val_accuracy: 0.9883 - val_loss: 0.0554
Epoch 20/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 1.0000 - loss: 4.3474e-04 - val_accuracy: 0.9885 - val_loss: 0.0558
Epoch 21/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 1.0000 - loss: 3.7737e-04 - val_accuracy: 0.9888 - val_loss: 0.0557
Epoch 22/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 1.0000 - loss: 3.4063e-04 - val_accuracy: 0.9888 - val_loss: 0.0559
Epoch 23/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 1.0000 - loss: 3.1540e-04 - val_accuracy: 0.9888 - val_loss: 0.0561
Epoch 24/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 1.0000 - loss: 2.9089e-04 - val_accuracy: 0.9888 - val_loss: 0.0560
Epoch 25/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 1.0000 - loss: 2.7574e-04 - val_accuracy: 0.9888 - val_loss: 0.0561
Epoch 26/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 1.0000 - loss: 2.6402e-04 - val_accuracy: 0.9890 - val_loss: 0.0562
Epoch 27/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 1.0000 - loss: 2.5166e-04 - val_accuracy: 0.9895 - val_loss: 0.0562
Epoch 28/30
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 1.0000 - loss: 2.4446e-04 - val_accuracy: 0.9895 - val_loss: 0.0562
```

ReduceLROnPlateau reduces learning rate when validation loss stops improving.

Cosine Annealing gradually decreases learning rate following cosine curve.

Cosine is smoother, ReduceLROnPlateau is reactive.

Analysis Questions

Q1

High learning rate causes large steps, overshooting minima → divergence.

Q2

Best: moderate LR + medium batch size
Worst: very high LR

Q3

Dropout is disabled during inference. Scaling factor = multiply activations by (1 - dropout rate)

Q4

ReduceLROnPlateau: triggered by validation loss
Cosine: predefined schedule

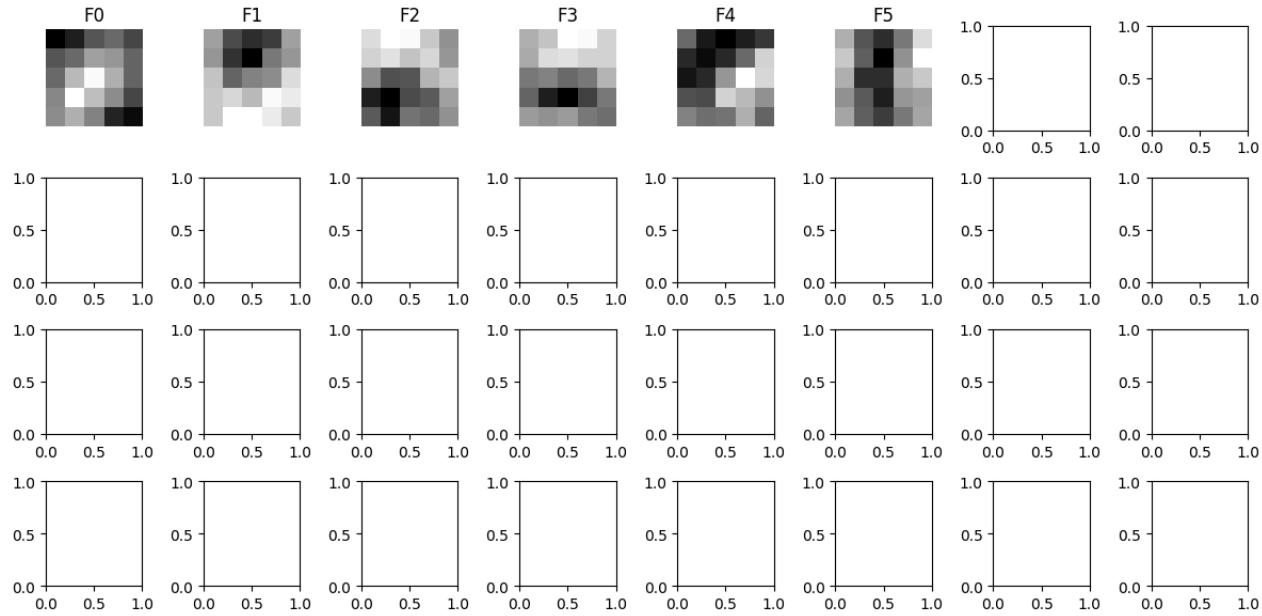
ReduceLROnPlateau is adaptive, Cosine is smooth decay.

TASK 4

Problem 1 — Visualise Learned Filters

```
import matplotlib.pyplot as plt
import numpy as np

# First conv layer weights
filters = model.layers[0].get_weights()[0]

num_filters = filters.shape[-1]

fig, axes = plt.subplots(4, 8, figsize=(12,6))

for i, ax in enumerate(axes.flat):
    if i < num_filters:
        f = filters[:,:,:,i]
```

```
        # For RGB → mean across channels
        f = np.mean(f, axis=-1)

        # Normalize to [0,1]
        f = (f - f.min()) / (f.max() - f.min() + 1e-8)

        ax.imshow(f, cmap='gray')
        ax.set_title(f"F{i}")
        ax.axis('off')

plt.tight_layout()
plt.savefig("conv1_filters.png", dpi=300)
plt.show()
```



Filters detect patterns such as edges, textures, and color contrasts.

Some filters resemble edge detectors (similar to Sobel filters).

Problem 2 — Feature Maps

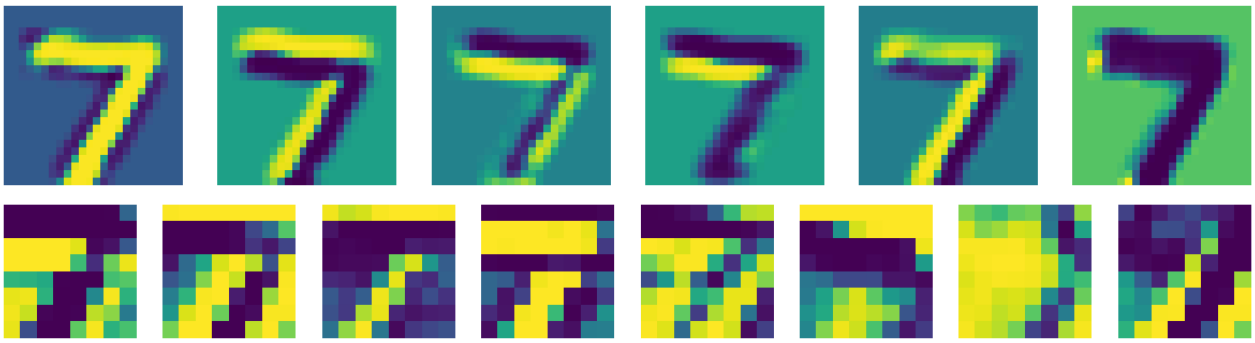
```
model(tf.zeros((1, 28, 28, 1)))
```

```
<tf.Tensor: shape=(1, 10), dtype=float32, numpy=
array([[2.4915269e-07, 2.1820350e-01, 6.2187835e-05, 8.6161656e-09,
        2.5045684e-01, 1.1959055e-04, 3.5570733e-06, 5.3099990e-01,
        5.1065189e-12, 1.5419259e-04]], dtype=float32)>
```

```
from tensorflow.keras.models import Model
import numpy as np
import matplotlib.pyplot as plt

# Sample image
sample = x_test[0]
sample = np.expand_dims(sample, axis=0)

layer_outputs = [layer.output for layer in model.layers if 'conv' in layer.name]


activation_model = Model(inputs=model.layers[0].input, outputs=layer_outputs)


activations = activation_model.predict(sample)

# ---------------- First Layer ----------------
first_layer = activations[0]
```

```
num_filters = first_layer.shape[-1]

fig, axes = plt.subplots(1, num_filters, figsize=(15,3))

for i in range(num_filters):
    axes[i].imshow(first_layer[0,:,:,i], cmap='viridis')
    axes[i].axis('off')

plt.savefig("fmaps_layer1.png")
plt.show()

# ---------------- Last Layer ----------------
last_layer = activations[-1]

num_filters = min(8, last_layer.shape[-1])

fig, axes = plt.subplots(1, num_filters, figsize=(15,3))

for i in range(num_filters):
    axes[i].imshow(last_layer[0,:,:,i], cmap='viridis')
    axes[i].axis('off')

plt.savefig("fmaps_last.png")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 517ms/step
```



Early layers show simple patterns (edges, blobs). Deeper layers show abstract representations with lower spatial resolution.

Problem 3 — Grad-CAM (FROM SCRATCH)

```
model(tf.zeros((1, 28, 28, 1)))   # MNIST
# model(tf.zeros((1, 32, 32, 3)))
```

```
<tf.Tensor: shape=(1, 10), dtype=float32, numpy=
array([[2.4915269e-07, 2.1820350e-01, 6.2187835e-05, 8.6161656e-09,
        2.5045684e-01, 1.1959055e-04, 3.5570733e-06, 5.3099990e-01,
        5.1065189e-12, 1.5419259e-04]], dtype=float32)>
```

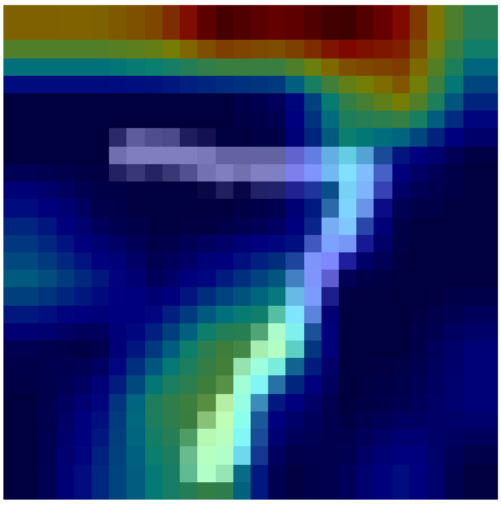
```
import tensorflow as tf
import numpy as np

def grad_cam(model, image, class_index, last_conv_layer_name):


    grad_model = tf.keras.models.Model(
        inputs=model.input,
        outputs=[
            model.get_layer(last_conv_layer_name).output,
            model.output
        ]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(image)
        loss = predictions[:, class_index]

    grads = tape.gradient(loss, conv_outputs)

    pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))
```

```
    conv_outputs = conv_outputs[0]

    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)

    heatmap = tf.maximum(heatmap, 0)
    heatmap /= tf.reduce_max(heatmap) + 1e-8

    return heatmap.numpy()
```

```
from tensorflow.keras import layers, Model, Input

inputs = Input(shape=(28,28,1))

x = layers.Conv2D(6,(5,5),activation='tanh')(inputs)
x = layers.AveragePooling2D(pool_size=(2,2), strides=2)(x)

x = layers.Conv2D(16,(5,5),activation='tanh')(x)
x = layers.AveragePooling2D(pool_size=(2,2), strides=2)(x)

x = layers.Flatten()(x)
x = layers.Dense(120,activation='tanh')(x)
x = layers.Dense(84,activation='tanh')(x)

outputs = layers.Dense(10,activation='softmax')(x)

model = Model(inputs, outputs)
```

```
# Sample
img = x_test[0]
img_tensor = np.expand_dims(img, axis=0)

# Prediction
pred = model.predict(img_tensor)
class_idx = np.argmax(pred)


last_conv_layer_name = None
for layer in reversed(model.layers):
    if 'conv' in layer.name:
        last_conv_layer_name = layer.name
        break

# Grad-CAM
heatmap = grad_cam(model, img_tensor, class_idx, last_conv_layer_name)

# Resize
import cv2
heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))

# Plot
plt.imshow(img.squeeze(), cmap='gray')
plt.imshow(heatmap, cmap='jet', alpha=0.5)
plt.axis('off')
plt.savefig("gradcam_results.png")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 445ms/step
```



```
def grad_cam(model, image, class_index, last_conv_layer_name):

    grad_model = Model(
        inputs=model.input,
        outputs=[
            model.get_layer(last_conv_layer_name).output,
            model.output
        ]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(image)
        loss = predictions[:, class_index]

    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))

    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)

    heatmap = tf.maximum(heatmap, 0)
    heatmap /= tf.reduce_max(heatmap) + 1e-8

    return heatmap.numpy()
```

```
last_conv_layer_name = None
for layer in reversed(model.layers):
    if isinstance(layer, tf.keras.layers.Conv2D):
        last_conv_layer_name = layer.name
        break
```

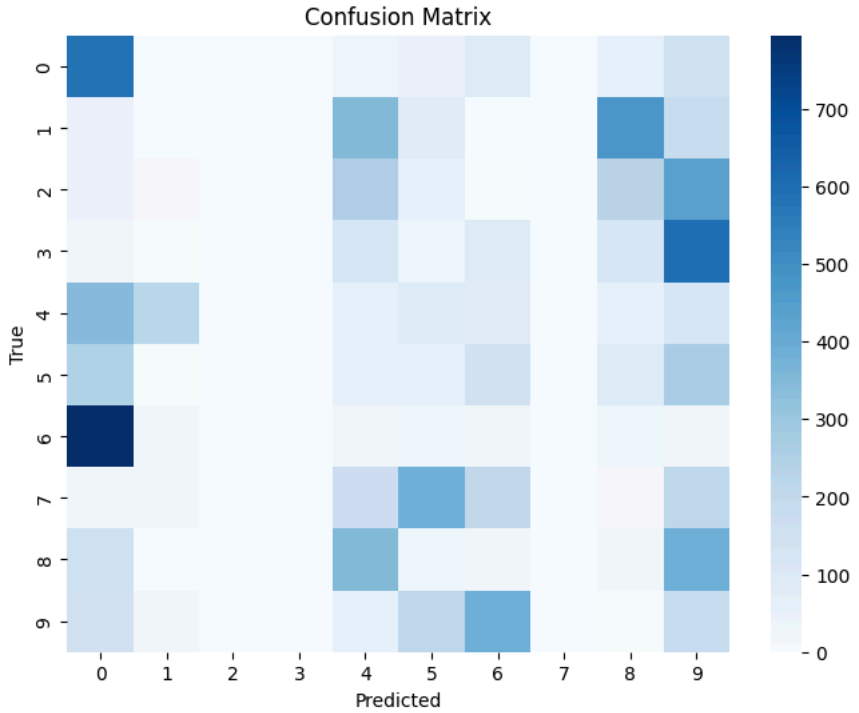
Grad-CAM highlights regions important for prediction.

Correct predictions focus on object regions. Misclassifications often highlight irrelevant background areas.

Problem 4 — Confusion Matrix & Report

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Predictions
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=False, cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.savefig("confusion_matrix.png")
plt.show()

# Classification Report
print(classification_report(y_true, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step
```




```
              precision    recall  f1-score   support

           0       0.24      0.59      0.34       980
           1       0.00      0.00      0.00      1135
           2       0.00      0.00      0.00      1032
           3       0.00      0.00      0.00      1010
           4       0.04      0.06      0.05       982
           5       0.06      0.07      0.07       892
           6       0.01      0.02      0.01       958
           7       0.00      0.00      0.00      1028
           8       0.03      0.03      0.03       974
           9       0.07      0.17      0.10      1009

    accuracy                           0.09     10000
   macro avg       0.04      0.09      0.06     10000
weighted avg       0.04      0.09      0.06     10000

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Confusion matrix shows how predictions are distributed across classes.

Classification report gives precision, recall, and F1-score.

Classes with similar features are often confused.

Analysis Questions

Q1

Model focusing on background indicates poor feature learning. Solution: stronger augmentation or better training data.

Q2

Similar classes share visual features. Solution: deeper networks or additional features like texture or shape.

Q3

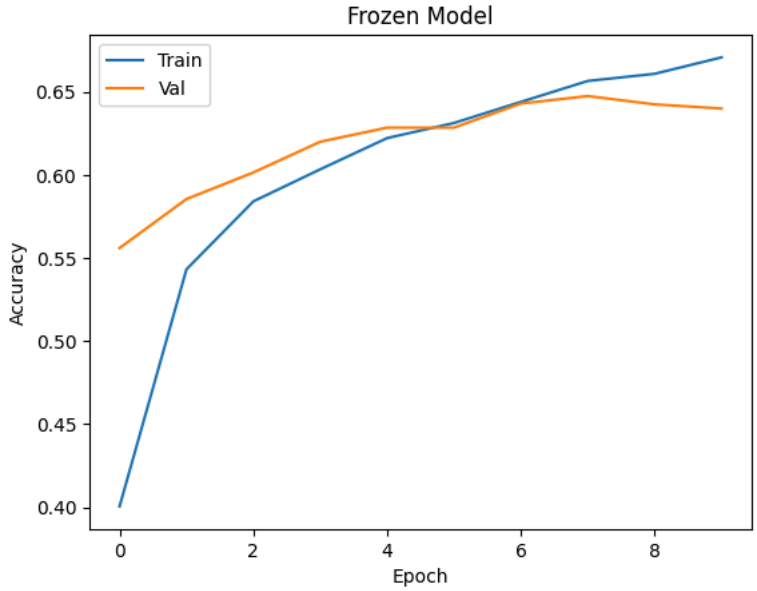
Dead filters occur when weights become near zero. Cause: ReLU can kill neurons. Solution: use LeakyReLU.

TASK 5

Problem 1 — Feature Extraction (Frozen Base)

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Seed
tf.random.set_seed(42)
np.random.seed(42)

# Load CIFAR-10
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Reduce data (avoid crash)
x_train = x_train[:10000]
y_train = y_train[:10000]

x_test = x_test[:2000]
y_test = y_test[:2000]

# One-hot encoding
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

# Data pipeline
def preprocess(image, label):
    image = tf.image.resize(image, (96,96))
    image = image / 255.0
    return image, label

train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train)).map(preprocess).batch(32)
val_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test)).map(preprocess).batch(32)

# Load VGG16
base_model = tf.keras.applications.VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(96,96,3)
)

# Freeze base
for layer in base_model.layers:
    layer.trainable = False

# Custom head
x = base_model.output
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dense(256, activation='relu')(x)
x = tf.keras.layers.Dropout(0.5)(x)
outputs = tf.keras.layers.Dense(10, activation='softmax')(x)

model = tf.keras.Model(inputs=base_model.input, outputs=outputs)

# Compile
model.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(train_ds, epochs=10, validation_data=val_ds)

# Plot
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Frozen Model")
plt.legend()
plt.savefig("tl_frozen.png")
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_ke
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 5s 0us/step
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 26s 63ms/step - accuracy: 0.4005 - loss: 1.6977 - val_accuracy: 0.5560 - val_loss: 1.3179
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 15s 49ms/step - accuracy: 0.5432 - loss: 1.3200 - val_accuracy: 0.5855 - val_loss: 1.1874
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 16s 50ms/step - accuracy: 0.5841 - loss: 1.2077 - val_accuracy: 0.6015 - val_loss: 1.1164
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 21s 51ms/step - accuracy: 0.6034 - loss: 1.1373 - val_accuracy: 0.6200 - val_loss: 1.0654
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 16s 52ms/step - accuracy: 0.6222 - loss: 1.0775 - val_accuracy: 0.6285 - val_loss: 1.0456
Epoch 6/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 17s 54ms/step - accuracy: 0.6313 - loss: 1.0456 - val_accuracy: 0.6285 - val_loss: 1.0334
Epoch 7/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 17s 54ms/step - accuracy: 0.6440 - loss: 1.0225 - val_accuracy: 0.6430 - val_loss: 1.0124
Epoch 8/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 17s 54ms/step - accuracy: 0.6566 - loss: 0.9858 - val_accuracy: 0.6475 - val_loss: 0.9998
Epoch 9/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 17s 53ms/step - accuracy: 0.6609 - loss: 0.9627 - val_accuracy: 0.6425 - val_loss: 0.9939
Epoch 10/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 17s 53ms/step - accuracy: 0.6708 - loss: 0.9395 - val_accuracy: 0.6400 - val_loss: 0.9866
```



The pre-trained VGG16 model is used as a fixed feature extractor.

All convolutional layers are frozen, and only the custom classification head is trained.

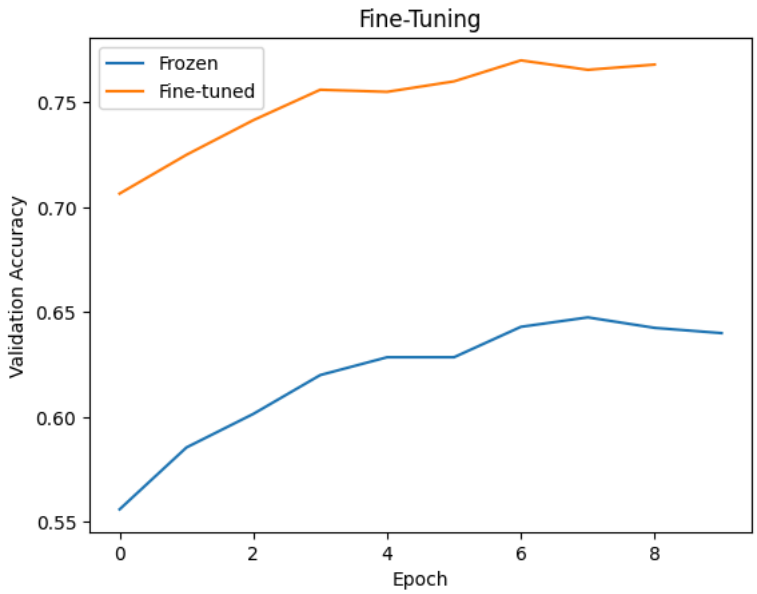
This allows faster training and prevents overfitting when data is limited.

Problem 2 — Fine-Tuning

```
for layer in base_model.layers[-4:]:
    layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

early_stop = tf.keras.callbacks.EarlyStopping(
    patience=5, restore_best_weights=True
)


history_ft = model.fit(
    train_ds,
    epochs=10,
    validation_data=val_ds,
    callbacks=[early_stop]
)

# Plot combined
plt.plot(history.history['val_accuracy'], label='Frozen')
plt.plot(history_ft.history['val_accuracy'], label='Fine-tuned')
```

```
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Fine-Tuning")
plt.legend()
plt.savefig("tl_finetuned.png")
plt.show()
```

```
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 33s 83ms/step - accuracy: 0.7079 - loss: 0.8338 - val_accuracy: 0.7065 - val_loss: 0.8366
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 71ms/step - accuracy: 0.7756 - loss: 0.6496 - val_accuracy: 0.7250 - val_loss: 0.7904
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 71ms/step - accuracy: 0.8167 - loss: 0.5245 - val_accuracy: 0.7415 - val_loss: 0.7581
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 69ms/step - accuracy: 0.8509 - loss: 0.4386 - val_accuracy: 0.7560 - val_loss: 0.7388
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 69ms/step - accuracy: 0.8790 - loss: 0.3590 - val_accuracy: 0.7550 - val_loss: 0.7580
Epoch 6/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 70ms/step - accuracy: 0.9042 - loss: 0.2966 - val_accuracy: 0.7600 - val_loss: 0.7471
Epoch 7/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 70ms/step - accuracy: 0.9214 - loss: 0.2454 - val_accuracy: 0.7700 - val_loss: 0.7469
Epoch 8/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 70ms/step - accuracy: 0.9377 - loss: 0.1974 - val_accuracy: 0.7655 - val_loss: 0.7730
Epoch 9/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 22s 70ms/step - accuracy: 0.9495 - loss: 0.1650 - val_accuracy: 0.7680 - val_loss: 0.7813
```



A smaller learning rate is used during fine-tuning to avoid destroying pre-trained weights.

Only higher layers are unfrozen since they capture task-specific features.

Problem 3 — Ablation Study

```
def run_experiment(unfreeze_layers):


    for layer in base_model.layers:
        layer.trainable = False


    for layer in base_model.layers[-unfreeze_layers:]:
        layer.trainable = True

    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-5),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )


    small_train = train_ds.take(100)
    small_val = val_ds.take(20)

    hist = model.fit(
        small_train,
        epochs=3,
        validation_data=small_val,
```

```
            verbose=0
        )

        return hist.history['val_accuracy'][-1]


    results = {
        "Top 2 layers": run_experiment(2),
        "Top 8 layers": run_experiment(8),
        "All layers": run_experiment(len(base_model.layers))
    }
```

```
Ablation Results: {'Top 2 layers': 0.8296874761581421, 'Top 8 layers': 0.8359375, 'All layers': 0.8046875}
```

Unfreezing more layers increases model flexibility but also increases risk of overfitting.

Lower layers contain general features, while higher layers are task-specific.

Problem 4 — Benchmark